



BIT1123/BISE2093/DIT1113

BIT1123 OBJECT ORIENTED PROGRAMMING

ASSIGNMENT 1 - INDIVIDUAL

(20%)

ALL SECTIONS

FACULTY OF INFORMATION TECHNOLOGY

CITY UNIVERSITY MALAYSIA

CYBERJAYA CAMPUS

LECTURER BY

SIR NAZMIRUL IZZAD BIN NASSIR

PREPARED BY:

No.	Name	ID	Program	Class code	Passport No.
1.	Ei Phyu Phyu Phway	(202505010374)	BCSSE	202605F0813	MJ915550

3. Self-Reflective Report

3.1 Introduction

The Object-Oriented Programming (OOP) course has provided me with an opportunity to develop my programming knowledge and understand the fundamental principles of Java programming. Throughout the tutorials from Week 1 to Week 10, I learned not only how to write Java programs but also how to organize source code, use Git and GitHub for version control, and develop a simple graphical user interface using Java Swing.

At the beginning of the tutorials, my understanding of Java programming was mainly focused on basic programming concepts and writing simple source code. As the tutorials progressed, I was introduced to important Object-Oriented Programming concepts such as classes and objects, constructors, inheritance, polymorphism, encapsulation, abstraction, and access modifiers. I also learned how Java can be used to create applications that store data using ArrayList, work with files, and respond to user interactions through a graphical user interface.

Another important part of this learning experience was becoming familiar with GitHub and repository management. I learned how to create and organize a repository, use Git commands such as `git add`, `git commit`, and `git push`, and arrange my tutorial work into separate folders. This helped me understand that programming is not only about writing code but also about maintaining an organized project and documenting the development process.

This report reflects on my learning journey throughout the ten weeks of tutorials. It discusses the knowledge I gained, challenges I encountered, how I solved those challenges, improvements in my Java programming skills, my understanding of OOP concepts, and my plans for further learning.

3.2 Knowledge Gained from Each Tutorial

3.2.1 Week 1 – Development Environment and GitHub

In Week 1, I learned how to set up my development environment using GitHub and Code spaces. This was an important starting point because I learned how programming files can be stored and managed using a repository rather than keeping all files only on my computer.

I practiced basic Git commands such as `git init`, `git add`, `git commit`, and `git push`. These commands helped me understand the basic process of saving changes and uploading them to GitHub.

I also created simple Java programs such as `HelloWorld.java` and `StudentGrade.java`. These programs helped me become familiar with Java source files and the basic process of creating and running Java programs.

The main knowledge I gained from Week 1 was that version control can help programmers organize their work and keep track of changes. I also became more comfortable with the Java development environment.

3.2.2 Week 2 – Classes, Objects and Constructors

In Week 2, I was introduced to the basic concepts of Object-Oriented Programming. I learned how to create Java classes and objects and how objects can be created from classes using the `new` keyword.

I also learned about constructors and how they can be used to initialize instance variables when an object is created. In addition, I practiced calling methods such as `displayInfo()` to allow objects to perform specific actions.

This tutorial helped me understand the relationship between a class and an object. A class can be viewed as a structure or blueprint, while an object represents an instance created from that class. Understanding this relationship gave me a stronger foundation for the OOP concepts that were introduced in later tutorials.

2.3 Week 3–4 – Inheritance and Polymorphism

During Week 3 and Week 4, I learned about inheritance and polymorphism. I worked with a `Person` base class and a `Student` derived class to understand how one class can inherit properties and methods from another class.

I learned how the `extends` keyword is used to create inheritance relationships. I also learned how the `super()` keyword can be used to initialize the constructor of a parent class.

Another important concept was method overriding. By using @Override, I was able to redefine a method from the parent class in the child class. This helped me understand the basic idea of polymorphism and how the same method can behave differently depending on the object or class using it.

This tutorial was important because inheritance and polymorphism are fundamental concepts of OOP. It also helped me understand how code can be reused instead of writing the same code repeatedly.

3.2.4 Week 5 – Encapsulation

In Week 5, I learned about encapsulation. The tutorial demonstrated how data can be protected inside a class by declaring fields as private.

I practiced creating getter and setter methods such as `getStudentID()`, `setStudentID()`, `getCGPA()`, and `setCGPA()`. These methods allowed controlled access to private data from outside the class.

Before learning encapsulation, I mainly focused on whether a program could produce the required output. After learning this concept, I started to understand that the way data is organized and accessed is also important.

Encapsulation showed me how OOP can be used to make programs more organized and protect the internal data of objects.

3.2.5 Week 6 – Access Modifiers and Class Relationships

In Week 6, I learned more about access modifiers and class relationships by working with an Employee parent class and a Lecturer subclass.

I learned about the protected access modifier and how it allows child classes to access certain fields while still restricting direct access from unrelated external classes.

This tutorial helped me understand that Java provides different levels of access control. I also became more aware of how access modifiers affect the relationship between classes.

The tutorial strengthened my understanding of inheritance and helped me see how access control can be used when designing classes.

3.2.6 Week 7 – Abstraction

In Week 7, I learned about abstraction through a Smart Home Appliance system. I created an abstract Appliance class containing common methods such as turnOn() and turnOff().

The main idea I learned was that abstraction allows common features to be defined while hiding unnecessary implementation details. Instead of focusing on every internal detail of an appliance, the user can interact with common operations such as turning an appliance on or off.

This tutorial helped me understand another important principle of OOP. I also learned how abstract classes and abstract methods can be used to provide a common structure for related classes.

3.2.7 Week 8–9 – ArrayList and File I/O

In Week 8 and Week 9, I learned how to store dynamic collections of data using Java ArrayList. This was different from working with individual variables because an ArrayList can store multiple items and can change in size.

I also learned File I/O operations by creating a To-Do List application. The application saved tasks into a task.txt file using BufferedWriter and read the saved tasks using BufferedReader.

Exception handling was also important because file operations can cause errors. I learned that programs should handle possible problems instead of assuming that every operation will always be successful.

This tutorial improved my understanding of how Java programs can work with external files and manage multiple pieces of data.

3.2.8 Week 10 – Java Swing GUI

In Week 10, I learned how to develop a simple Graphical User Interface using Java Swing. I created a Programming Quiz Battle application using components such as JFrame, JButton, and JLabel.

I also learned how event handling works through ActionListener and actionPerformed(). This allowed the program to respond when users interacted with buttons.

This was one of the most interesting tutorials because I was able to see how Java can be used to create an interactive application instead of only producing output through the console.

The Week 10 tutorial also helped me understand the connection between programming logic and user interaction.

3.3 Challenges Encountered

Throughout the tutorials, I encountered several challenges while learning Java and OOP. One of the main challenges was understanding the differences between the various OOP concepts. In particular, inheritance, polymorphism, encapsulation, and abstraction were sometimes difficult to distinguish because they are related to how classes and objects are designed.

Another challenge was understanding constructors and the relationship between parent and child classes. When working with inheritance, I needed to understand how super() works and how information from a parent class can be used by a child class.

Access modifiers were another challenging area. Understanding the differences between private, public, and protected required me to pay attention to which classes were allowed to access specific data.

I also found File I/O more challenging because it involved additional classes such as BufferedWriter and BufferedReader, as well as exception handling. There were more steps involved compared with writing a simple console program.

GitHub and Git commands were also challenging at the beginning. I had to understand the difference between making changes to files locally and pushing those changes to the remote repository.

Finally, developing the Java Swing GUI required me to understand event handling and how buttons interact with program logic. This was more complex than the earlier console-based programs.

3.4. How the Challenges Were Overcome

I overcame these challenges by practicing the examples from the tutorials and reviewing the source code carefully. Instead of trying to memorize every concept, I focused on understanding what each part of the code was doing.

For OOP concepts, I compared the different concepts with the classes and examples used in the tutorials. For example, I studied how `extends` represents inheritance, how `@Override` is used for method overriding, how private fields and getters/setters demonstrate encapsulation, and how abstract classes demonstrate abstraction.

When I encountered problems with Java syntax or program structure, I checked the code carefully and compared it with the tutorial examples. I also practiced running the programs after making changes so that I could identify errors and understand how changes affected the output.

For GitHub, repeated practice with commands such as `git add`, `git commit`, and `git push` helped me become more comfortable with version control. Organizing my work into separate folders also helped me understand how a programming repository should be structured.

For File I/O, I focused on understanding the purpose of each class and method instead of trying to memorize the complete code. Similarly, for Java Swing, I learned how GUI components and event handlers work together.

These experiences taught me that programming skills improve through continuous practice, debugging, and learning from mistakes.

3.5. Improvements in Java Programming Skills

My Java programming skills improved significantly throughout the ten tutorials. At the beginning, I mainly worked with simple Java programs. By the end of the tutorials, I was able to work with classes, objects, constructors, inheritance, polymorphism, encapsulation, abstraction, ArrayList, File I/O, exception handling, and GUI components.

One of the most important improvements was my ability to organize code into classes. I became more comfortable creating separate classes for different responsibilities rather than putting all the code into a single file.

I also improved my understanding of Java syntax and program structure. I became more familiar with creating methods, passing information between classes, creating objects, and controlling access to data.

My debugging skills also improved. When a program did not work as expected, I became more willing to examine the code and identify where the problem occurred rather than immediately assuming that the entire program was incorrect.

Another improvement was my ability to use GitHub as part of my programming workflow. I learned that maintaining a repository can help organize programming projects and demonstrate development progress.

Overall, these tutorials helped me become more confident in writing and understanding Java programs.

3.6. Understanding of Object-Oriented Programming Concepts

The tutorials helped me develop a clearer and more practical understanding of the major Object-Oriented Programming (OOP) concepts in Java. Instead of only learning the definitions of OOP concepts, I was able to apply them through different Java programs and examples throughout the tutorials. The main concepts I learned were classes and objects, encapsulation, inheritance, polymorphism, abstraction, and access modifiers.

3.6.1 Classes and Objects

I learned that a class provides the structure or blueprint for creating objects, while an object is an instance of a class. This concept was introduced in Week 2 and became the foundation for understanding the other OOP concepts.

For example, I created Java classes such as Student and then created objects from those classes using the new keyword. I also learned how objects can contain their own data and use methods defined inside their class. Constructors were also introduced to initialize the values of objects when they are created.

This helped me understand that classes allow related data and behaviors to be organized together. Instead of writing everything in one program, I can divide the program into different classes that represent different objects or responsibilities.

3.6.2 Encapsulation

I learned that encapsulation involves protecting the data of an object by controlling how it can be accessed and modified. In Week 5, I practiced declaring fields as private and using public getter and setter methods to access them.

For example, methods such as `getStudentID()`, `setStudentID()`, `getCGPA()`, and `setCGPA()` were used to access and modify student information. This helped me understand that the data of an object does not always need to be directly accessible from outside the class.

Encapsulation is useful because it provides better control over data and helps make a program more organized. Through this tutorial, I learned that protecting data is an important part of designing a good OOP program.

3.6.3 Inheritance

I learned that inheritance allows one class to inherit properties and methods from another class. This concept was introduced through the `Person` and `Student` classes in Weeks 3–4 and was further developed through the `Employee` and `Lecturer` classes in Week 6.

The `extends` keyword is used to create an inheritance relationship between a parent class and a child class. For example, a `Student` can inherit common characteristics from a `Person`, while a `Lecturer` can inherit common employee information from an `Employee`.

Inheritance helped me understand the importance of code reuse. Instead of creating the same variables and methods again in every class, common features can be placed in a parent class and reused by related child classes.

3.36.4 Polymorphism

I learned about polymorphism mainly through method overriding. In Weeks 3–4, I practiced overriding methods from a parent class in a child class using the `@Override` annotation.

This helped me understand that a child class can provide its own implementation of a method that it inherits from its parent class. Therefore, the same method can behave differently depending on the class that provides the implementation.

Learning polymorphism helped me understand how OOP can make programs more flexible. It also showed me that inheritance is not only about sharing code but can also allow related classes to behave differently when necessary.

3.6.5 Abstraction

I learned that abstraction allows important common features to be defined while hiding unnecessary implementation details. This concept was demonstrated in Week 7 through the Smart Home Appliance system.

I worked with an abstract Appliance class that defined common operations such as turnOn() and turnOff(). The purpose was to provide a common structure for different appliances without requiring the main program to focus on the internal implementation of every appliance.

This helped me understand that abstraction allows programmers to focus on what an object should do rather than always focusing on how every operation is implemented. It also showed me how abstract classes can provide a common design for related classes.

3.6.6 Access Modifiers

I also developed a better understanding of access modifiers such as private, public, and protected. These modifiers determine how classes, variables, and methods can be accessed.

I learned that private members provide restricted access within their class, while public members can be accessed more widely. In Week 6, I also learned about protected, which is useful when working with inheritance because it allows subclasses to access certain members of a parent class.

Understanding access modifiers helped me see how Java provides different levels of access control. It also showed me that choosing the appropriate access modifier is important when designing classes and protecting data.

3.6.7 Relationship Between the OOP Concepts

As I progressed through the tutorials, I realized that these OOP concepts are not completely separate from each other. They can work together when developing a Java application.

For example, a program can use **classes and objects** as its basic structure, **encapsulation** to protect data, **inheritance** to reuse common features, **polymorphism** to allow different implementations of related methods, and **abstraction** to hide unnecessary implementation details. **Access modifiers** then help control how the different parts of the program can interact with each other.

Understanding how these concepts work together helped me move from simply learning individual Java features to understanding how Object-Oriented Programming can be used as an overall approach to software design.

3.6.8 Overall Understanding

Overall, the tutorials significantly improved my understanding of Object-Oriented Programming. At first, concepts such as inheritance, polymorphism, encapsulation, and abstraction seemed difficult because they were new to me. However, practicing them through different Java examples helped me understand their purposes and differences.

I now understand that OOP is not simply about creating classes. It is a programming approach that organizes data and behavior into objects and establishes relationships between classes. The concepts I learned throughout these tutorials have provided me with a strong foundation for developing more structured, reusable, and maintainable Java programs.

These OOP principles will also be useful in my future programming projects because they can help me design programs in a more organized way rather than putting all the program logic into a single class or file.

3.7 Future Learning Plans

In the future, I plan to continue improving my Java programming skills by creating more practical projects. Rather than only following tutorial examples, I want to practice designing small applications independently.

One of my main goals is to become more confident with advanced Object-Oriented Programming concepts and learn how to apply them when designing larger programs. I also want to improve my understanding of Java collections, file handling, exception handling, and GUI development.

I plan to practice Git and GitHub more regularly so that version control becomes a normal part of my programming workflow. I would also like to improve my repository documentation and learn how to maintain projects in a more professional way.

Another area I want to explore is developing larger Java applications that combine multiple concepts together. For example, I would like to create applications that use multiple classes, inheritance, encapsulation, ArrayList, file storage, and GUI components in one project.

In addition, I plan to improve my debugging and problem-solving skills. Instead of depending heavily on examples, I want to become more independent when identifying errors, understanding why they occur, and developing appropriate solutions.

These plans will help me build a stronger foundation for future programming subjects and software development projects.

3.8. Conclusion

In conclusion, the BIT1123 Object-Oriented Programming course has been an important learning experience that helped me develop both my Java programming knowledge and my understanding of software development practices. Throughout Week 1 to Week 10, I gradually progressed from writing simple Java programs to developing programs that use different Object-Oriented Programming concepts, data collections, file handling, and graphical user interfaces. Each tutorial introduced a new concept and helped me build upon the knowledge gained from the previous tutorials.

At the beginning of the course, I focused mainly on understanding basic Java syntax and learning how to run simple programs. Through Week 1, I also learned how Git and GitHub can be used to manage programming work and maintain different versions of a project. This was valuable because it introduced me to a development workflow that is more organized than simply saving individual source code files. By organizing my tutorial work into separate folders and maintaining them in one GitHub repository, I became more aware of the importance of project organization and documentation.

As the tutorials progressed, I developed a stronger understanding of Object-Oriented Programming. Learning about classes, objects, and constructors in Week 2 gave me the foundation required to understand more advanced OOP concepts. The lessons on inheritance and polymorphism in Weeks 3–4 helped me understand how classes can be related and how existing code can be reused. Week 5 introduced encapsulation and showed me the importance of protecting data using private fields and controlling access through getter and setter methods. Week 6 further developed my understanding of class relationships and access modifiers, while Week 7 helped me understand abstraction through the Smart Home Appliance example.

The later tutorials allowed me to apply Java in more practical situations. In Weeks 8–9, I learned how to use ArrayList to manage collections of data and how to save and retrieve information using File I/O. These concepts showed me how Java applications can work with data beyond simple console output. In Week 10, I developed a Java Swing GUI application using components such as JFrame, JButton, and JLabel, together with event handling

through ActionListener. This was particularly useful because it demonstrated how programming logic can be connected to an interactive user interface.

During this learning journey, I also faced several challenges. Some OOP concepts were difficult to understand initially because inheritance, polymorphism, encapsulation, and abstraction are closely related. I also needed time to become comfortable with constructors, access modifiers, File I/O, exception handling, and GUI event handling. However, these difficulties became valuable learning experiences. By practicing the tutorial examples, reviewing my code, running programs repeatedly, and identifying errors, I gradually became more confident in solving programming problems.

Another important achievement from this assignment was improving my ability to organize and document my programming work. The assignment required one GitHub repository containing the tutorial source codes, README documentation, and the self-reflective report. This helped me understand that a software project should not only contain working code but should also be structured clearly so that other people can understand and navigate it. Preparing the README also encouraged me to summarize what I learned from each tutorial and explain how the different parts of my repository were organized.

Overall, this course has improved my confidence in Java programming and given me a stronger foundation in Object-Oriented Programming. I now have a better understanding of how classes and objects can be designed, how OOP principles can improve code organization and reuse, and how different Java features can be combined to create functional applications. More importantly, I have learned that becoming a better programmer requires continuous practice, debugging, patience, and willingness to learn from mistakes.

In the future, I plan to build on the knowledge gained from these tutorials by developing larger and more independent Java projects. I would like to combine multiple OOP concepts with collections, File I/O, GUI development, and other programming techniques in practical applications. I also plan to continue using GitHub to manage my projects and improve my documentation and version-control skills. By applying the knowledge from this course to future projects, I hope to become more independent and confident in solving programming problems and eventually apply these skills to real-world software development.

Ultimately, the ten tutorials have provided me with more than just knowledge of Java syntax. They have helped me develop a better understanding of programming logic, Object-Oriented Programming, project organization, version control, debugging, and reflective learning. Although there is still much more for me to learn, this course has provided a strong foundation for my future studies in Computer Science and Software Engineering.

3.9. GitHub Repository

GitHub Repository:

[eiphyuphyuphway834-hue/Ei_Phyu_Phyu_Phway](https://github.com/eiphyuphyuphway834-hue/Ei_Phyu_Phyu_Phway)

The repository contains the tutorial source codes from Week 1 to Week 10, the README documentation, and this self-reflective report. The repository is organized according to the tutorial structure so that the source code and supporting files can be easily located.